# Queues

# QUEUE

- Queue Likes the stack,
- the queue is a list-like structure that provides restricted access to its elements.
- Queue elements
  - inserting at the back (called an **enqueue** operation)
  - removing from the front (called a **dequeue** operation).
- Queues operate like standing in line at a movie theater ticket counter.
  - If nobody cheats, then newcomers go to the back of the line.
- The person at the front of the line is the next to be served.
- Thus, queues release their elements in order of arrival.

# QUEUE

- They call a queue a “FIFO” list, which stands for “First-In, First-Out.”
- The following Figure 4.24 shows a sample queue ADT.
- This section presents two implementations for queues:
  1. the array-based queue
  2. the linked queue.

# Queues

FIFO: First in, First Out

Restricted form of list: Insert at one end, remove from the other.

Notation:

- Insert: Enqueue
- Delete: Dequeue
- First element: Front
- Last element: Rear

# /** Queue ADT */ Figure 4.24

```
public interface Queue {
/** Reinitialize the queue. The user is responsible for reclaiming the storage used by the queue elements. */
  public void clear();
/** Place an element at the rear of the queue. @param it The element being enqueued. */
    public void enqueue(E it);
/** Remove and return element at the front of the queue. @return The element at the front of the queue. */
  public E dequeue();
/** @return The front element. */ public E frontValue(); /** @return The number of elements in the queue. */
 public int length(); }
```

Figure 4.24 The Java ADT for a queue

# Array based

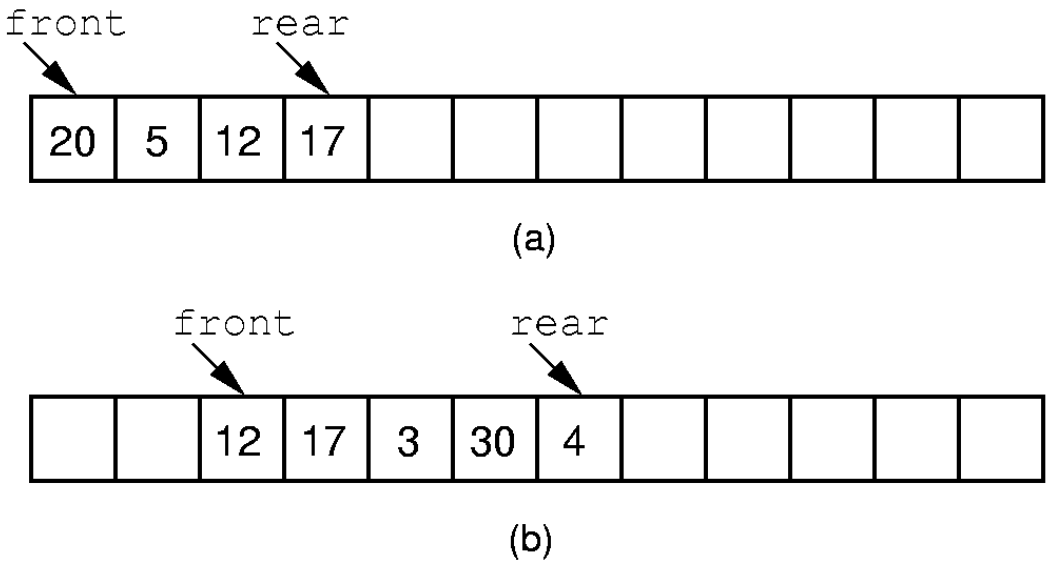
The queue:

# 4.3.1 Array-Based Queues

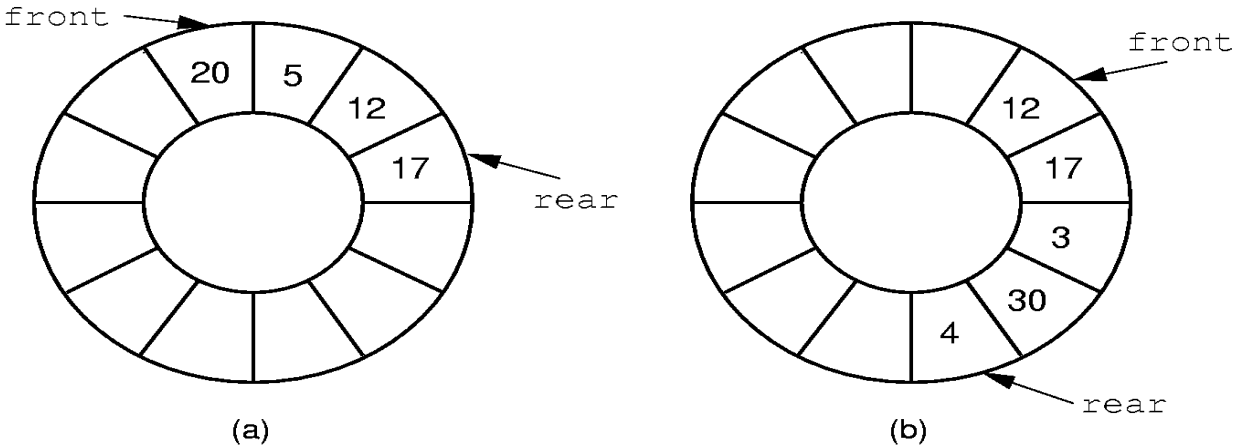
- The array-based queue is somewhat tricky to implement effectively.
- A simple conversion of the array-based list implementation is not efficient.

# Queue Implementation (1)

- ways of management array-based queue:
- 1-the queue elements be at the beginning n elements of the array, then either enqueue or dequeue must be Θ(n).
- 2- to let the elements “drift” within the array.



# Queue Implementation (2)




By using the mod function, we can easily achieve the effect of a "circular" queue.
This leaves one more issue:

1) Where do the front and rear pointers go ?
2) how do we distinguish a full from an empty queue?

- Given a fixed position for the front element (and its pointer), there are n-1 possible states for the queue (0 through n elements in the queue for an array of size n), but only n possible positions for rear.
- To solve this dilemma,
  - we must either leave an empty slot in the queue,
  - or use and external variable to determine if the queue is empty or not.

# /** Array-based queue implementation */

```
class AQueue<E> implements Queue<E> {
    private static final int defaultSize = 10;
    private int maxSize; // Maximum size of queue
    private int front; // Index of front element
    private int rear; // Index of rear element
    Private int size;
    private E[] listArray; // Array holding queue elements
    /** Constructors */
    AQueue() { this(defaultSize); }
    AQueue(int size) {
        maxSize = size+1; // One extra space is allocated
        rear = 0; front = 1;
        listArray = (E[])new Object[maxSize]; // Create listArray
        size=0;
}
```

# clear

```
/** Reinitialize */
public void clear()
{
    rear = 0; front = 1;
 }
```

**Sol2**

```
public void clear()
{
        size=0;
 }
```

# ENQUEUE

```
/** Put "it" in queue */
public void enqueue(E it) {
    assert ((rear+2) % maxSize) != front : "Queue is full";
    rear = (rear+1) % maxSize; // Circular increment
    listArray[rear] = it;
}
```

**Sol2:**

```
/** Put "it" in queue */
public void enqueue(E it) {
    assert (size!=maxSize): "Queue is full";
    rear = (rear+1) % maxSize; // Circular increment
    listArray[rear] = it;
   size++;
}
```

# DEQUEUE

```
/** Remove and return front value */
public E dequeue() {
    assert length() != 0 : "Queue is empty";
     E it = listArray[front];
     front = (front+1) % maxSize; // Circular increment
     return it;
}
```

**Sol2:**

```
public E dequeue() {
    assert size!= 0 : "Queue is empty";
     E it = listArray[front];
     front = (front+1) % maxSize; // Circular increment
     Size--;
     return it;
}
```

# frontValue

```
/** @return Front value */
public E frontValue() {
    assert length() != 0 : "Queue is empty";
    return listArray[front];
}
```

# length

```
/** @return Queue size */
public int length(){
    return ((rear+maxSize) - front + 1) % maxSize;
}
/*Sol2:*/
public int length(){
    return size;
}
```

# Queue

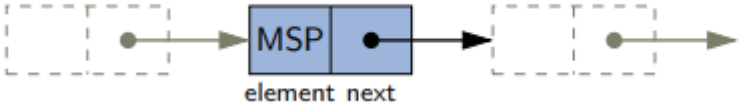
Linked based

# Linked queue

- The linked queue implementation is a straightforward adaptation of the linked list.
- Figure 4.28 shows the linked queue class declaration. Methods front and rear are pointers to the front and rear queue elements, respectively.
- We will use a header link node, which allows for a simpler implementation of the enqueue operation by avoiding any special cases when the queue is empty.
- On initialization, the front and rear pointers will point to the header node, and front will always point to the header node while rear points to the true last link node in the queue.
- Method enqueue places the new element in a link node at the end of the linked list (i.e., the node that rear points to)
- and then advances rear to point to the new link node.
- Method dequeue removes and returns the first element of the list.

# /** Linked queue implementation */

```
class LQueue<E> implements Queue<E> {
    private Link<E> front; // Pointer to front queue node
    private Link<E> rear; // Pointer to rear queuenode
    private int size; // Number of elements in queue
    /** Constructors */
    public LQueue() { init(); }
    public LQueue(int size) { init(); } // Ignore size
    /** Initialize queue */
    private void init() {
    front = rear = new Link<E>(null);
    size = 0;
}
```



- **/** Reinitialize queue */**
- **public void clear() { init(); }**

# enqueue

```
/** Put element on rear */
public void enqueue(E it) {
    rear.setNext(new Link<E>(it, null));
    rear = rear.next();
    size++;
}
```

# dequeue

```
/** Remove and return element from front */
public E dequeue() {
    assert size != 0 : "Queue is empty";
    E it = front.next().element(); // Store dequeued value
    front.setNext(front.next().next()); // Advance front
    if (front.next() == null) rear = front; // Last Object
    size--;
    return it; // Return Object
}
```

# /** @return Front element */

- **public E frontValue() {**
- **assert size != 0 : "Queue is empty";**
- **return front.next().element();}**

# /** @return Queue size */

- public int length() { return size; } theta(1)
- public int length() { // thete(n)
- int size=0;
- private Link<E> temp=front;
- While(temp.next !=rear){
- temp=temp.next();
- size++;
- }
- Retuen size;
- }

# 4.3.3 Comparison of Array-Based and Linked Queues

- All member functions for both the array-based and linked queue implementations require constant time.
- The space comparison issues are the same as for the equivalent stack implementations.